

Entregable 2 — Manual Operativo de Datos

Esquema de tablas vinculadas (Airtable) + esquemas JSON de transferencia entre integraciones — Proyecto ADC Triage

Airtable funciona como la **memoria persistente** del sistema: es la única fuente de verdad sobre en qué estado está cada consulta. n8n no guarda estado propio en ningún momento — lee y escribe siempre contra estas tablas, que están vinculadas entre sí por relaciones (*Linked Records*) para que ningún dato quede aislado.

1. Mapa de relaciones entre tablas

Tabla	Rol	Relacionada con
Consultas	Tabla central del triage — una fila por email entrante, con su ciclo de vida completo (Estado).	→ Base de Conocimiento RAG (directrices usadas para redactar) → Log de Errores (errores que ocurrieron durante su procesamiento)
Base de Conocimiento RAG	Directrices privadas y atómicas de ADC. El nodo de redacción de IA consulta esto en vez de usar internet.	← Consultas (múltiples consultas pueden citar la misma directriz)
Log de Errores	Registro relacional de fallos (API de IA, Airtable/Gmail/Slack, datos incompletos, timeout).	← Consultas (cada error apunta a la consulta que lo originó)
Centro de Comando - Pipeline de Contenido	Tabla de otro proceso del mismo workspace (pipeline de generación de contenido, Pre-entrega 6/7) — no participa del flujo de triage.	Vinculada a Base de Conocimiento RAG (comparte la misma base de directrices)

2. Esquema de campos por tabla

2.1 — Consultas (tbl ...F8DcJ8 — cerebro del triage)

Campo	Tipo	Notas
Remitente	email	Dirección del emisor del email original.
Asunto	singleLineText	Asunto del email.
Mensaje Original	multilineText	Cuerpo del email tal cual llegó.
Message ID	singleLineText	ID único de Gmail — usado para detectar duplicados.
Categoría	singleSelect	Comercial · Soporte Tecnico · Otro (la asigna la IA clasificadora).
Prioridad	singleSelect	Alta · Media · Baja (la asigna la IA clasificadora).
Estado	singleSelect	Pendiente → Procesado por IA → Aprobado por Humano → Enviado · Error · Descartado.
Respuesta Generada	multilineText	Texto redactado por la IA, pendiente de aprobación humana.
Notas de Revision	multilineText	Motivo si el revisor humano rechaza la respuesta.
Log de Error	multilineText	Resumen legible del último error, si lo hubo.

Campo	Tipo	Notas
Fecha de Recepcion	dateTime	Timestamp de creación de la fila (ISO).
Directrices RAG	multipleRecordLinks	Link → Base de Conocimiento RAG usada para redactar.
Contexto RAG (texto)	rollup	Rollup automático del contenido de las directrices linkeadas.
Log de Errores	multipleRecordLinks	Link → filas de Log de Errores asociadas a esta consulta.

2.2 — Base de Conocimiento RAG

Campo	Tipo	Notas
Tema	singleLineText	Campo primario — un tema por registro (directriz atómica).
Contenido	multilineText	Texto de la directriz — es lo único que la IA puede usar para redactar.
Categoría	singleSelect	Permite filtrar el RAG por la misma categoría que clasificó la IA.
Consultas	multipleRecordLinks	Link inverso → consultas que citaron esta directriz.

2.3 — Log de Errores

Campo	Tipo	Notas
Resumen	singleLineText	Título corto del error, generado dinámicamente (ej. "Fallo IA clasificando: <asunto>").
Tipo de Error	singleSelect	Fallo API IA · Datos Incompletos · Timeout · Otro (fallos de Airtable/Gmail/Slack).
Detalle	multilineText	Mensaje de error crudo: \$json.error.message o el JSON completo si no hay mensaje.
Fecha	dateTime	Timestamp del error (ISO).
Resuelto	checkbox	Marcado manual una vez que alguien revisó el error.
Consulta Relacionada	multipleRecordLinks	Link → la fila de Consultas que originó el error.

3. Esquemas JSON de transferencia entre integraciones

Estos son los payloads reales que viajan entre nodos del workflow de n8n. Se muestran simplificados (sin metadata interna de n8n) para que quede claro qué dato entra y qué dato sale en cada integración.

3.1 — Gmail Trigger → Normalizar Datos del Email (nodo Set)

```
{
  "remitente": "={{ $json.from.value[0].address }}",
  "asunto":    "={{ $json.subject }}",
  "mensaje":   "={{ $json.text }}",
  "message_id": "={{ $json.id }}",
  "thread_id": "={{ $json.threadId }}"
}
```

3.2 — IA Clasificar (Groq Llama-3.1-8B) → salida estructurada (JSON Schema forzado)

```
{
  "categoria": "Comercial | Soporte Tecnico | Otro",
  "prioridad": "Alta | Media | Baja"
}
// Ejemplo real: { "categoria": "Comercial", "prioridad": "Media" }
```

3.3 — Normalizar + Clasificación → Airtable “Consultas” (create / update)

```
{
  "Remitente":      "<remitente>",
  "Asunto":         "<asunto>",
  "Mensaje Original": "<mensaje>",
  "Message ID":     "<message_id>",
  "Fecha de Recepcion": "<timestamp ISO>",
  "Estado":         "Pendiente"
}
// luego, tras clasificar y redactar (update, matchingColumns: ["id"]):
{
  "Respuesta Generada": "<output del agente redactor>",
  "Categoria":          "<categoria>",
  "Prioridad":          "<prioridad>",
  "Estado":             "Procesado por IA"
}
```

3.4 — Airtable “Base de Conocimiento RAG” → Aggregate → IA Redactar

```
{
  "contexto_rag": ["<Contenido directriz 1>", "<Contenido directriz 2>", "..."]
}
// se concatena con \n\n y se inyecta en el prompt del agente redactor
```

3.5 — Slack Send & Wait (HITL) → respuesta del humano

```
{
  "data": {
    "approved": true
  }
}
// true → Marcar Aprobado por Humano → Enviar Gmail
// false → Marcar Rechazado por Humano → Estado = Descartado
```

3.6 — Cualquier nodo crítico (onError) → Airtable “Log de Errores”

```
{
  "Resumen":        "Fallo IA clasificando: <asunto>",
  "Tipo de Error":  "Fallo API IA",
  "Detalle":        "={{ $json.error?.message ?? JSON.stringify($json) }}"
}
```

```
"Fecha": "<timestamp ISO>",  
"Consulta Relacionada": ["<record id de la fila en Consultas>"]  
}
```

Nota: los nombres de campo que ve n8n en pantalla ("Asunto", "Estado", etc.) se traducen internamente a IDs de columna de Airtable (ej. fldmYSK5CFEQ22zTB) — se muestran aquí con su nombre legible para que el documento sea entendible sin tener que abrir la base.